

Bitcoins and Cryptocurrencies

Note: most of the slides used in this course are derived from those available for the book “Bitcoins and Cryptocurrencies Technologies – A Comprehensive Introduction”, Arvind Narayanan, Joseph Bonneau, Edward Felten, Andrew Miller & Steven Goldfeder, 2016, Princeton University Press.

Lecture 1

Intro to Crypto and Cryptocurrencies

This lecture

- Background on cryptographic primitives useful for building “crypto”currencies
 - Hash functions
 - Properties of hash functions we are interested in
 - Applications that make use of these properties
 - Data structures that use hash functions
 - Digital signatures
 - Public Keys and Identities
- Two simple cryptocurrencies (precursors to Bitcoins)
 - Goofycoin
 - Scroogecoin

Cryptographic Hash Functions

What are Hash functions?

- Mathematical functions: A mapping of items (values) in the domain to items (values) in the range.
- A hash function is a **mathematical function** that **converts** any **digital data** into an **output string** with a **fixed number of characters**. Hashing is the one-way act of converting the data (called a **message**) into the output (called the **hash**).
- Hash functions are special mathematical functions that satisfy the following three properties:
 - **Inputs** (or items in the domain) can be **any size (not-fixed)**; technically size of input is not unbounded in practice
 - **Outputs** (or items in the range) are **fixed-size** (we'll generally employ a hash function such as **SHA-256** that has an output size of **256 bits**)
 - Efficiently computable, i.e., the mapping should be efficiently (in **polynomial time in terms of the input size**) computable

Cryptographically Secure Hash functions?

- In cryptography (and cryptography based applications such as bitcoins), we are interested in a special type of hash function, often referred as a **cryptographically secure hash functions**
- **What is a cryptographically secure hash function?**
 - Satisfies the following additional security properties:
 1. Collision Resistance
 2. Hiding or Pre-image Resistance
 3. Puzzle-friendliness

Property 1: Collision Resistance

- **Strong Collision Resistance:**

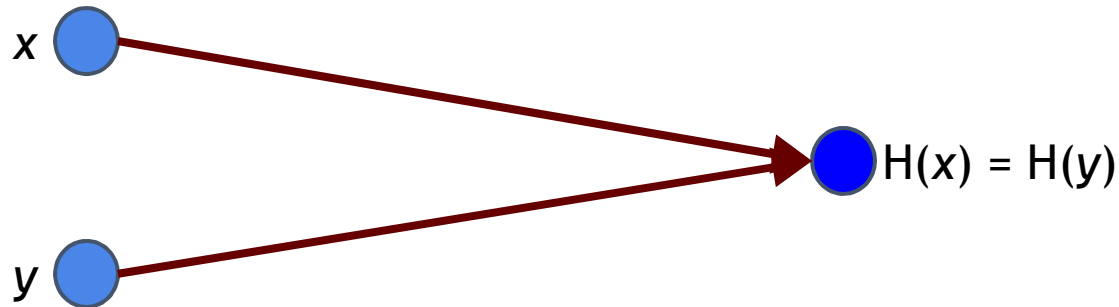
- A hash function is said to have strong collision resistance if it is **computationally infeasible to find any two distinct inputs that produce the same hash output**.
- In other words, it is extremely difficult to find any collision at all. Strong collision resistance provides a high level of security against adversaries attempting to find collisions.

- **Weak Collision Resistance (or second pre-image resistance):**

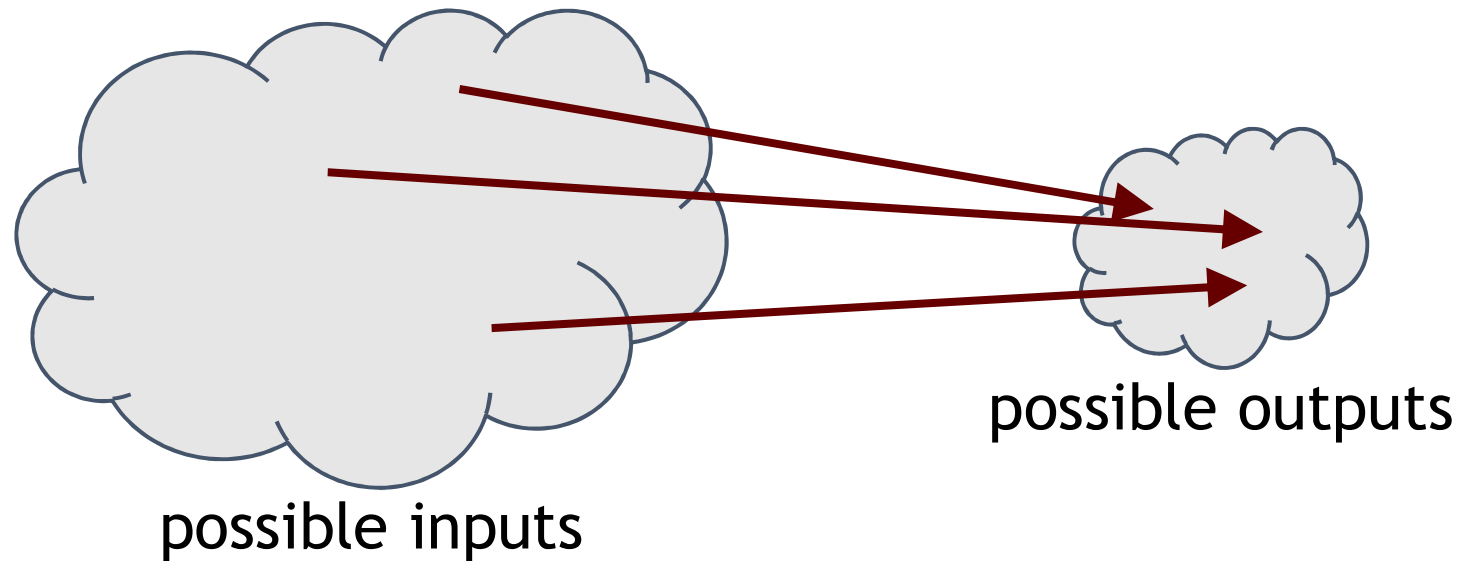
- A hash function is considered to have weak collision resistance if given an input message, it is **computationally infeasible to find any other input that produces the same hash output as the original message**.
- In this case, the original message is known, and the goal is to find a different input that hashes to the same value. Weak collision resistance is generally easier to achieve compared to strong collision resistance.

Property 1: Collision Resistance

- **Strong collision resistance:** Infeasible to find x and y such that $x \neq y$ and $H(x) = H(y)$
- **Weak collision resistance:** Given x and $H(x)$, it is infeasible to find $y \neq x$ such that $H(y) = H(x)$



Property 1: Collision Resistance



- For all hash functions (cryptographically secure or not) → Collisions exist
- For cryptographically secure hash functions → Difficult to find collisions!

Property 1: Collision Resistance

- How to find a collision in a Secure Hash function with a 256 bit output?
 - Strategy 1: Brute-force – Continue to randomly pick inputs and compute its Hash until you find a collision.
 - How long does this take?
 - Worst-case - $2^{256} + 1$ inputs
 - On average – more than 50% chance of finding collision after 2^{128} inputs (Birthday paradox)
 - More than 99.8% chance of collision after 2^{130} randomly chosen inputs
 - Brute-force always works, no matter what H is, in finding collision. However it takes too long to matter (2^{128} is a lot of tries!)
 - Strategy 2: Find cryptographic or other weaknesses in hash functions
 - Is the following function cryptographically secure $H(x) = x \bmod 2^3$? Yes/No? Why?
 - Most cryptographically secure has functions also have weaknesses. E.g., MD5 was considered to be secure, until after many years of research collisions were found. SHA 256 (currently used secure hash function) has no known attacks, but we don't know it is secure!

No Hash function has proven to be collision resistant!

Application of Collision-resistance: Hash as message digest

- Message digest: Output of a hash function is also called a message digest
- Now, if H is a secure hash function and if we know $H(x) = H(y)$, is it safe to assume that $x = y$? Why?
 - Yes! Because if the above is not true, it violates the collision-resistance property and H would not be secure!
- Application of message digests?
 - Verify integrity of large files
 - To verify integrity, rather than comparing files just compare hashes or message digests!
 - Useful because message digest or hash is smaller compared to inputs!

A hash serves as a fixed-length digest, or unambiguous summary, of a message!

Property 2: Hiding

- **Pre-image Resistance:**

- A hash function is said to have pre-image resistance if it is computationally **infeasible to find any input that maps to a specific hash output**.
- In other words, given a hash value, **it should be difficult to determine the original input that produced that hash**. Pre-image resistance ensures that the hash function's output provides no information about the input.

Property 2: Hiding

- **Hiding or pre-image resistance:** Given $H(x)$, it is infeasible to find x
- The property cannot be true in the form stated – Consider a simple example:



$H(\text{"heads"})$

easy to find x !

$H(\text{"tails"})$

Why?

Input x is not spread out or uniformly distributed, $P(x=\text{"heads"}) = 0.5$,
 $P(x=\text{"tails"}) = 0.5$, $P(\text{all other } x) = 0$!

Property 2: Hiding

- So how to achieve hiding or pre-image in this simple case?



$H(\text{"heads"})$

easy to find x !

$H(\text{"tails"})$

Why?

Combine the input x which is not spread out (or not uniformly distributed) with another input r which is spread out or uniformly distributed!

Property 2: Hiding

- **Hiding property (more precisely):**

If r is chosen from a probability distribution that has *high min-entropy* (*the possible outcomes of the random variable are difficult to predict or guess*), then given $H(r \mid x)$, it is infeasible to find x .

- **Min-entropy:** Entropy of a probability distribution function captures the **predictability** of the output of the function!

- Let r be a random variable which takes values from the set $\{1, 2, 3, \dots, n\}$, where n is very large such as 2^{256} , according to some probability distribution function f .
- If f is high min-entropy $\rightarrow$ distribution of the value of r is “very spread out” or “uniform”.
- In other words, r takes each value $1, 2, \dots, n$ with probability exactly $1/n$. If n is very large (e.g., 2^{256}), the probability of correctly guessing which value it took is very small (negligible)!

Application of hiding: Commitments

- Commit to a value, reveal it later.
- Analogy: Want to “seal a value in an envelope”, and “open the envelope” later.
- A commitment scheme consists of two algorithms:
- We require that the following two security properties hold:
 - **Hiding** : Given com , it is infeasible to find msg
 - **Binding** : It is infeasible to find two pairs $(msg, nonce)$ and $(msg', nonce')$ such that $msg \neq msg'$ and $commit(msg, nonce) == commit(msg', nonce')$ - **The binding property of a hash function ensures that it is computationally infeasible to find two different inputs that produce the same hash output.**

Application of hiding: Commitment API

$com := \text{commit}(msg, nonce)$ - *The commit function takes a message and secret random value, called a nonce, as input and returns a commitment.*
 $match := \text{verify}(com, msg, nonce)$ - *The verify function takes a commitment, nonce, and message as input. It returns true if $com == \text{commit}(msg, nonce)$ and false otherwise.*

To seal msg in envelope:

1. Select a random $nonce$ and keep it secret
2. Compute the commitment $com := \text{commit}(msg, nonce)$
3. Publish com

To open envelope:

1. Publish $nonce, msg$
2. Anyone can use $\text{verify}()$ to check validity of msg and the previously published com

Application of hiding: Security Properties of Commitment APIs

$com := \text{commit}(msg, nonce)$
 $match := \text{verify}(com, msg, nonce)$

Security properties:

1. **Hiding**: Given com (and if $nonce$ is chosen from a distribution with high min-entropy), infeasible to find msg .
2. **Binding**: Infeasible to find $\langle msg', nonce' \rangle \neq \langle msg, nonce \rangle$ such that:
 $\text{verify}(\text{commit}(msg, nonce), msg', nonce') == \text{true}$

So, how to implement a commitment scheme such that these two properties hold?

Application of hiding: Implementing Commitment APIs using Hash functions

$\text{commit}(msg, nonce) := H(nonce \mid msg)$

where *nonce* is a random 256-bit value
& *H* is a cryptographically secure hash
function

$\text{verify}(com, msg, nonce) := (H(nonce \mid msg) == com)$

Security properties:

Hiding or pre-image resistant property of secure Hash function

1. **Hiding:** Given $H(nonce \mid msg)$, infeasible to find *msg*.
2. **Binding:** Infeasible to find $\langle msg', nonce' \rangle \neq \langle msg, nonce \rangle$
such that

Collision-resistance property of secure Hash function

$$H(nonce' \mid msg') == H(nonce \mid msg)$$

Property 3: Puzzle-friendliness

- **Puzzle-friendliness property**: For every possible n -bit output value y , if k is chosen from a distribution with high min-entropy, then it is infeasible to find x such that $H(k \parallel x) = y$ in time significantly less than 2^n .

Intuition: If you want to target a Hash function H to have a particular output value y , and if part of the input (i.e., k) is chosen in a suitably randomized fashion, then its very difficult to find the other part of the input x to exactly hit the target output value (y)

Application of Puzzle-friendliness: Search puzzle

What is a search puzzle?

Given: A “puzzle ID” id (from high min-entropy distrib.), and a target set Y :

Objective: Try to find a “solution” x such that $H(id \mid x) \in Y$.

Puzzle-friendly property implies that no solving strategy is much better than trying random values of x .

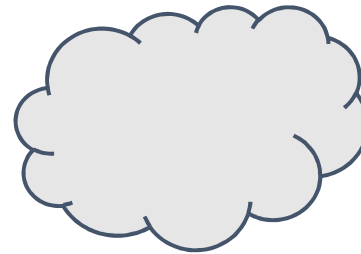


$Y=\{y_1\}$

Most time consuming!



$Y=\{y_1, y_2, y_3\}$



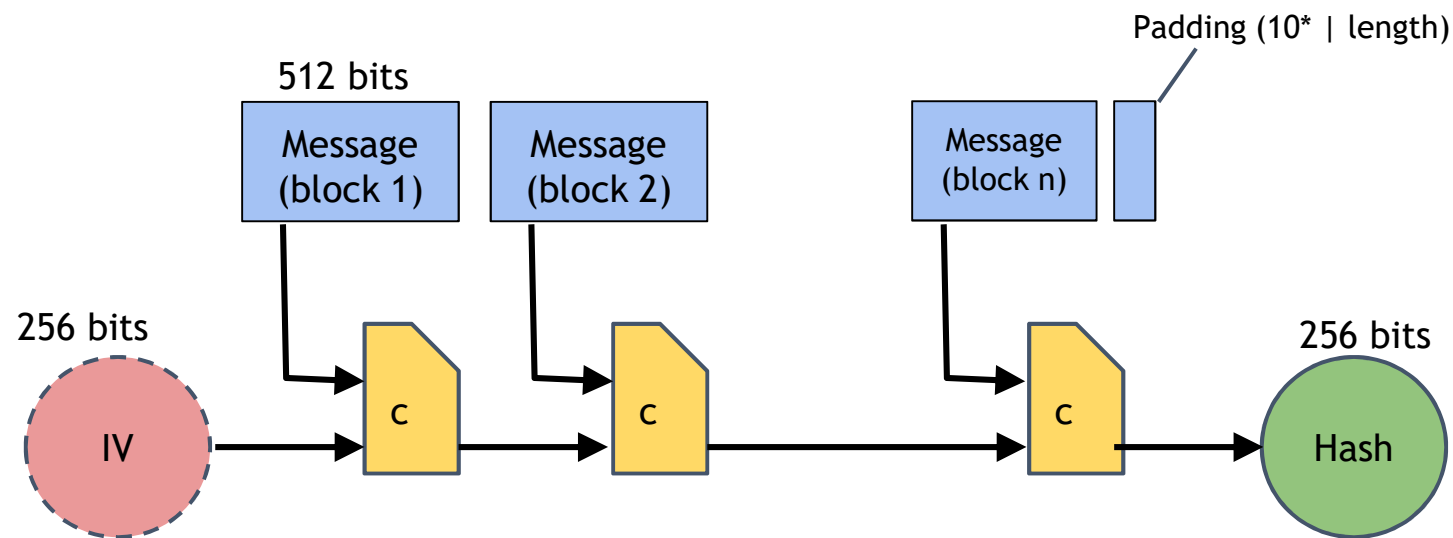
$Y=\{y_1, y_2, y_3, y_2^{256}\}$

Trivial!

Strength of the puzzle depends on the size of Y

Construction of Hash functions

- Hash functions are typically constructed from fixed-input compression functions!
 - Example: See construction of SHA-256 Hash function → SHA-256 used in Bitcoins
 - Also referred to as Merkle-Damgård Transform
- Why does it work?
 - Theorem: If c is collision-free, then SHA-256 is collision-free.

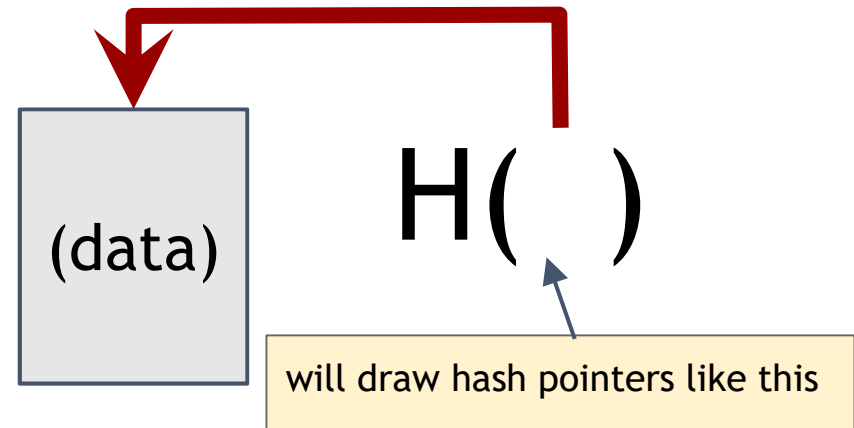


Hash Pointers and Data Structures

Hash Pointers

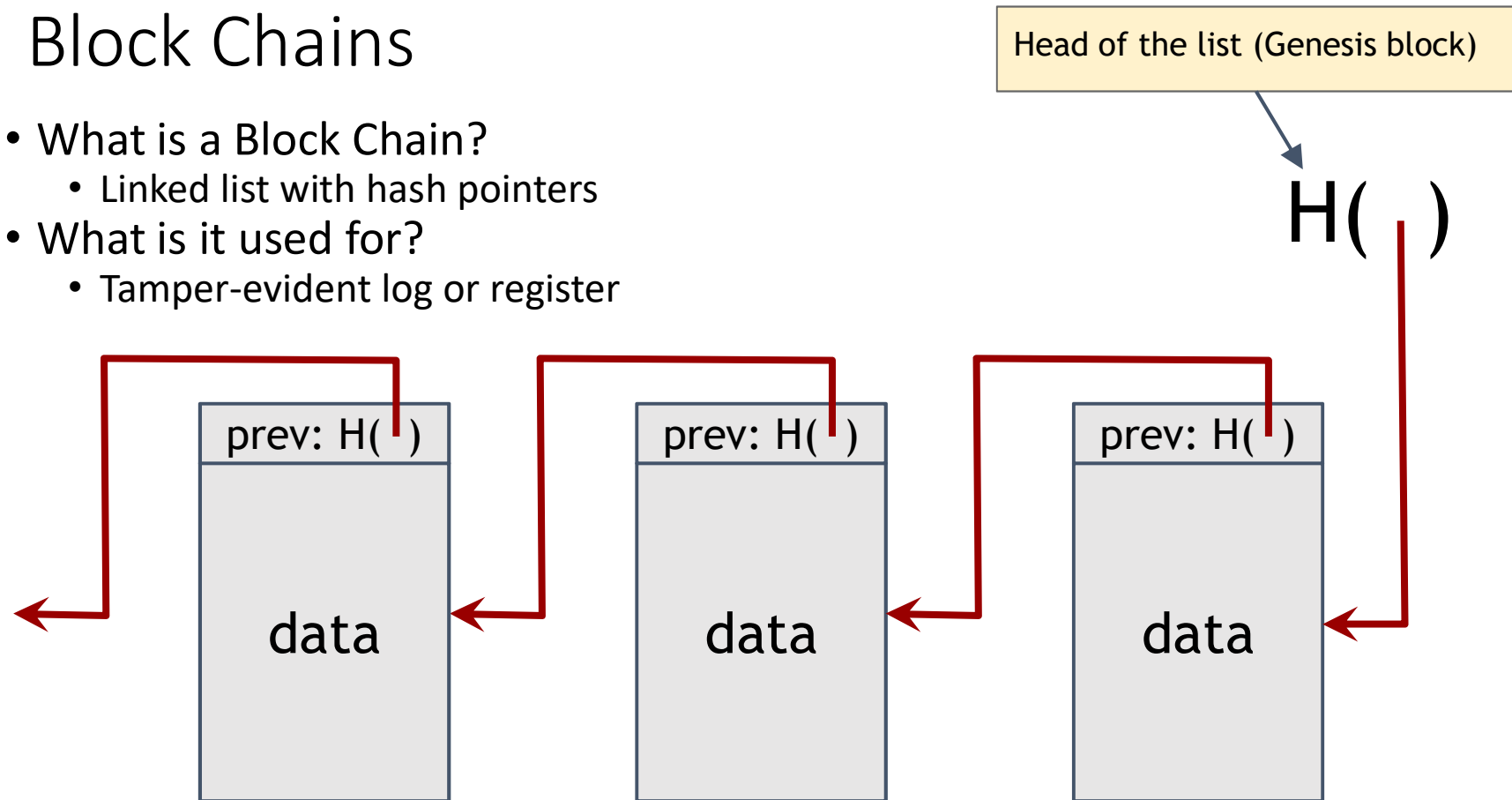
- Hash pointer is:
 - Pointer to where some info/data is stored, and
 - (Cryptographic) hash of the info
- What can you do with a hash pointer?
 - Retrieve or get back the info/data
 - Verify that the info/data hasn't changed
 - What else?

Use hash pointers to build data structures!



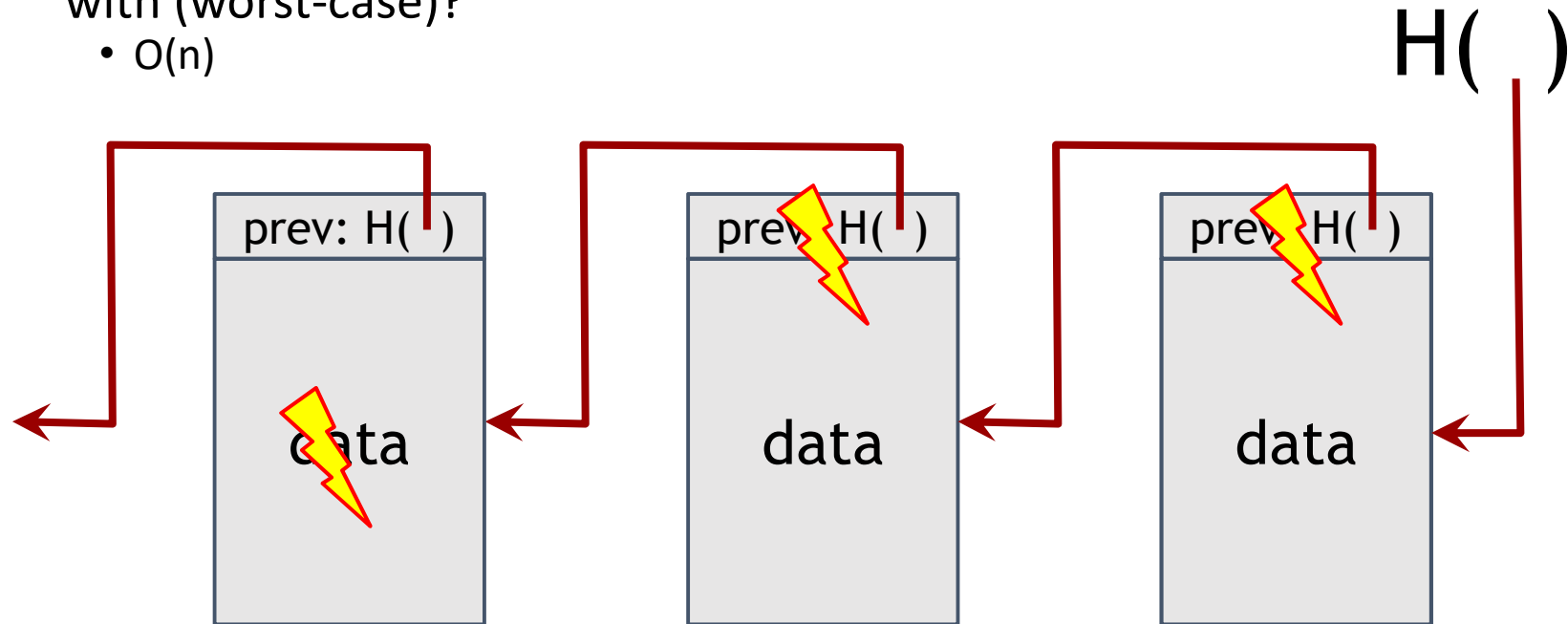
Block Chains

- What is a Block Chain?
 - Linked list with hash pointers
- What is it used for?
 - Tamper-evident log or register



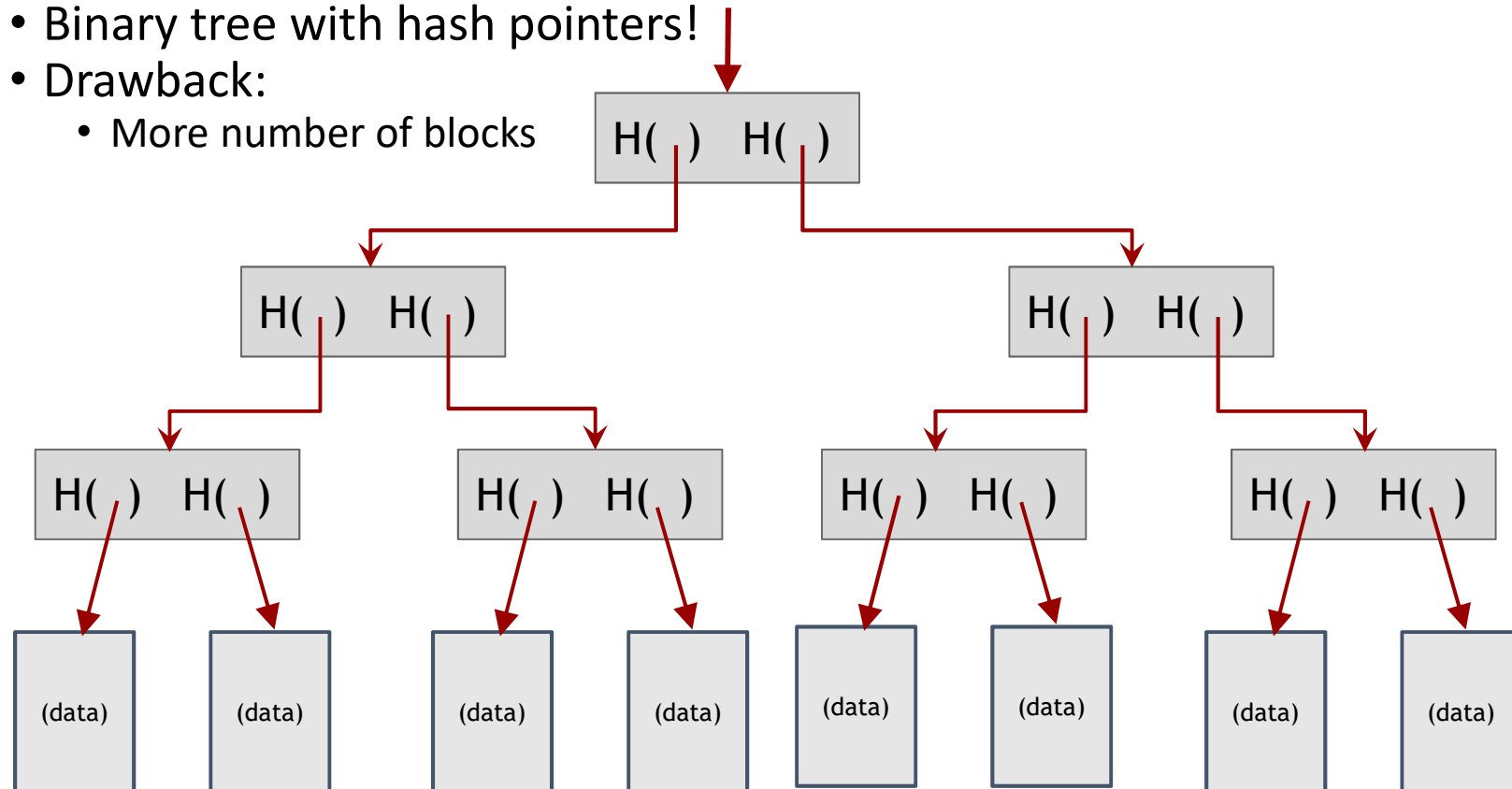
Tamper-evident Log

- What is a Tamper-evident log and how to detect tampering?
- How many verifications to prove that some data has not been tampered with (worst-case)?
 - $O(n)$



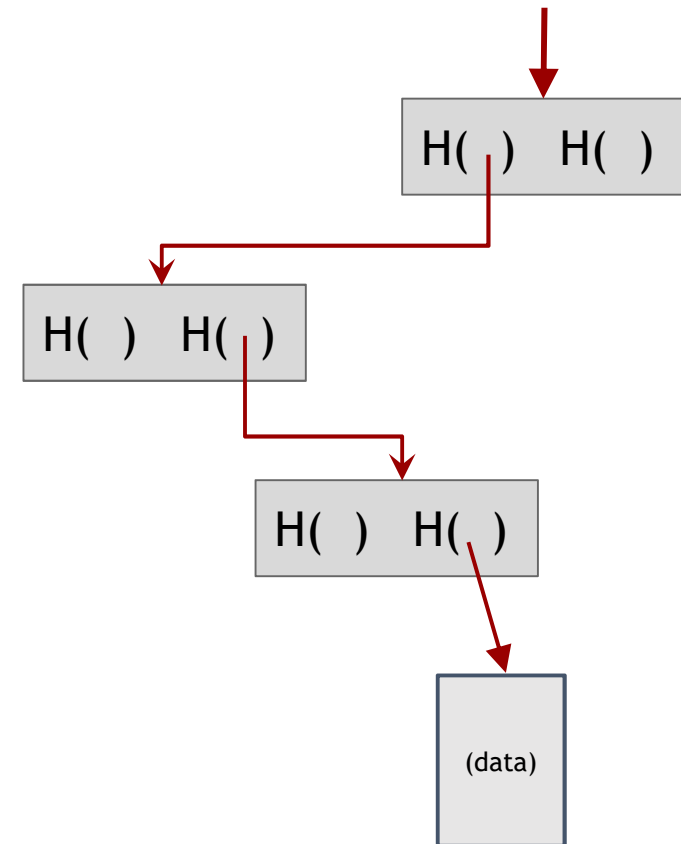
Merkle tree

- Binary tree with hash pointers!
- Drawback:
 - More number of blocks



Merkle tree

- Advantage:
 - Proving membership of a data block in the tree is easy
 - Only need to show $O(\log n)$ items
 - In other words, membership verification in $O(\log n)$ time/space
- How to prove non-membership?
 - Sorted Merkle trees: Order leafs of the tree in some fashion, say lexicographically, numerically, etc.
 - Verify membership of data before and after the missing one!
 - Non-membership verification also takes $O(\log n)$ time/space



More generally ...

**Hash pointers can be used in any pointer-based data structure
that has no cycles**

Digital Signatures

Digital Signatures

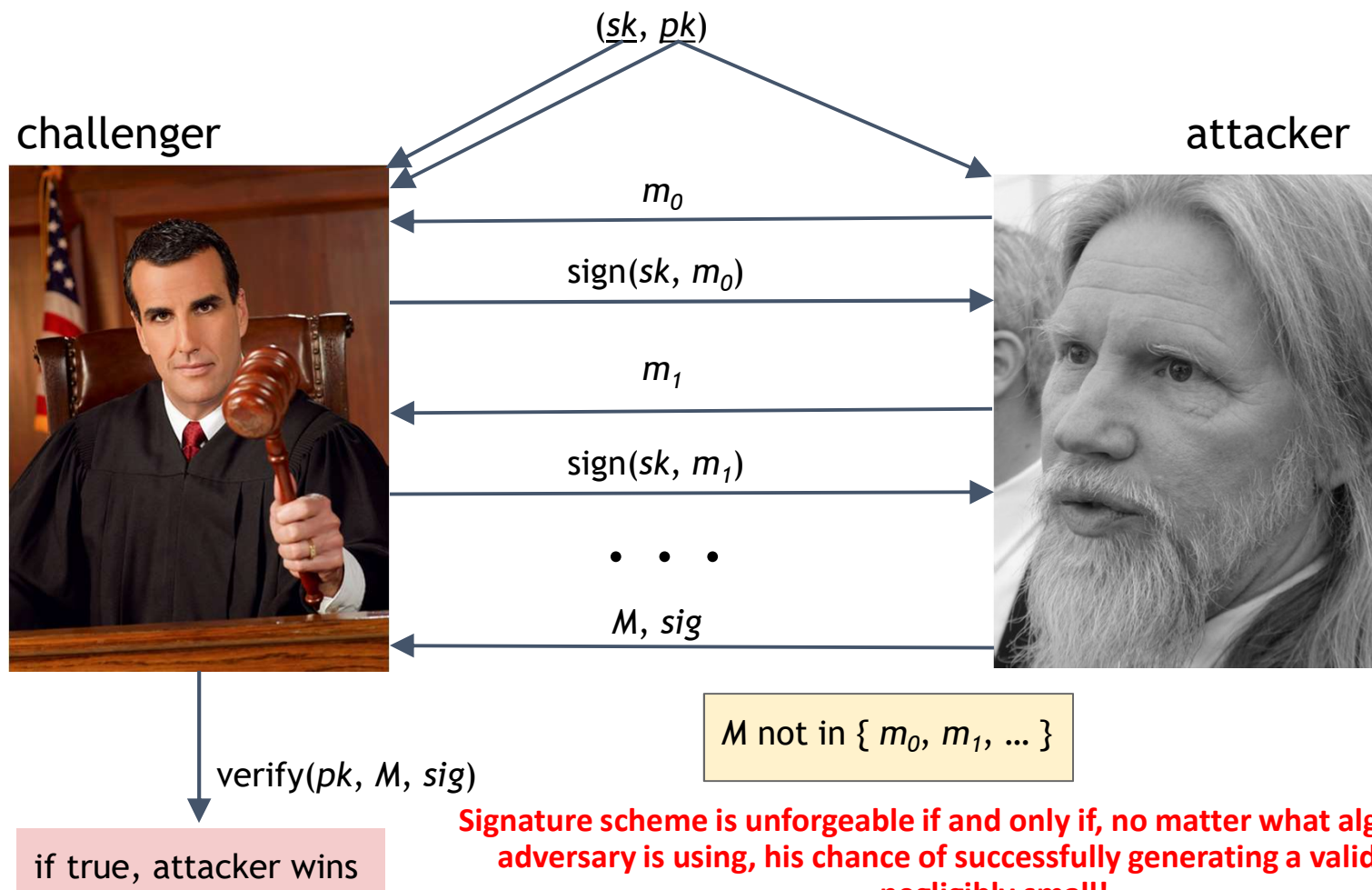
- Second cryptographic primitive (in addition to Hash functions) that we will need to build cryptocurrencies (and bitcoins)
- What are the properties we need from digital signatures? – same as properties we need from handwritten signatures
 - Only you can sign, but anyone can verify (**security**)
 - Signature tied to a particular document - can't be cut-and-pasted to another document (**unforgeability**)

Digital Signatures APIs

1. $(sk, pk) := \text{generateKeys}(\text{keysize})$
 sk : secret signing key
 pk : public verification key
 2. $sig := \text{sign}(sk, \text{message})$
 3. $\text{isValid} := \text{verify}(pk, \text{message}, sig)$
- can be randomized algorithms
- Is a deterministic algorithm

Digital Signatures Requirements

1. Valid signatures must always verify correctly
 - i.e., $\text{verify}(pk, \text{message}, \text{sign}(sk, \text{message})) == \text{true}$
 - Basic property for signatures to be useful!
2. Signatures should be existentially unforgeable $\rightarrow$ can't forge signatures
 - i.e., adversary who knows pk and gets to see signatures on messages of his choice, still can't produce a verifiable signature on another message
 - Can be formalized by means of the unforgeability game described next



Digital Signatures - Practical Concerns

- Many digital signature algorithms are randomized (esp. ones used in cryptocurrencies)
 - Need good source of randomness
 - Bad randomness → Even an otherwise secure signature algorithm is not secure
- Signature algorithms have fixed sized inputs → How to sign large messages (whose size is greater than input size of the algorithm)?
 - Use Hash(message) rather than message
- How to sign the entire Block chain?
 - Sign the entire hash pointer of the head block!
 - This signature “covers” the whole block chain structure

Digital Signatures used by Bitcoins

- Bitcoin uses Elliptic Curve Digital Signature Algorithm (ECDSA) standard
 - ECDSA is a US Government standard
- Bitcoin uses ECDSA over the standard elliptic curve secp256k1 → this curve is rarely used outside Bitcoins
 - Provides 128 bit of security (equivalent to performing 2^{128} symmetric encryptions)
 - Private key – 256 bits
 - Public key compressed – 257 bits
 - Message to be signed – 256 bits
 - Signature – 512 bits
- Technical details of ECDSA will be skipped here
- Good randomness is essential for ECDSA
 - If you foul this up in `generateKeys()` or `sign()` → you probably leaked your private key

Public Keys as Identities

Identities in a cryptocurrency

- Can we use public key pk , as generated before by `generateKeys(keysize)`, as an identity?
- For example, if you see signature sig such that `verify(pk, msg, sig) == true`, think of it as: pk says, “[msg]”.
- But, pk by itself cannot be used as an identity! To “speak for” pk , you must know matching secret key sk

Identities in a cryptocurrency

How to make a new identity?

- Create a new, random key-pair (sk , pk)
 - pk is the public “name” you can use [usually better to use $H(pk)$]
 - sk lets you “speak for” the identity
- You control the identity pk , because only you know sk
- Even if pk “looks random” that’s fine, nobody needs to know your real identity for the cryptocurrency application
- Just like while spending an actual currency note

Identities in a cryptocurrency

Decentralized identity management

- Anybody can make a new identity at any time make as many as you want!
- No central point of coordination
- These identities are called “addresses” in Bitcoin

Identities in a cryptocurrency

Privacy

- Addresses not directly connected to real-world identity
- But observer can link together an address's activity over time, make inferences
- Later: a whole chapter on privacy in Bitcoins ...

Simple Cryptocurrencies



GoofyCoin

GoofyCoin - Rules

- The **first rule** is that a **designated entity, Goofy**, can create new coins whenever he wants and these newly created coins belong to him.
- The **second rule** of Goofycoin is that **whoever owns a coin can transfer it** to someone else.
 - Transferring a coin is not simply a matter of sending the coin data structure to the recipient—it's done using cryptographic operations.

GoofyCoin – Coin Creation

Goofy can create new coins

signed by sk_{Goofy}

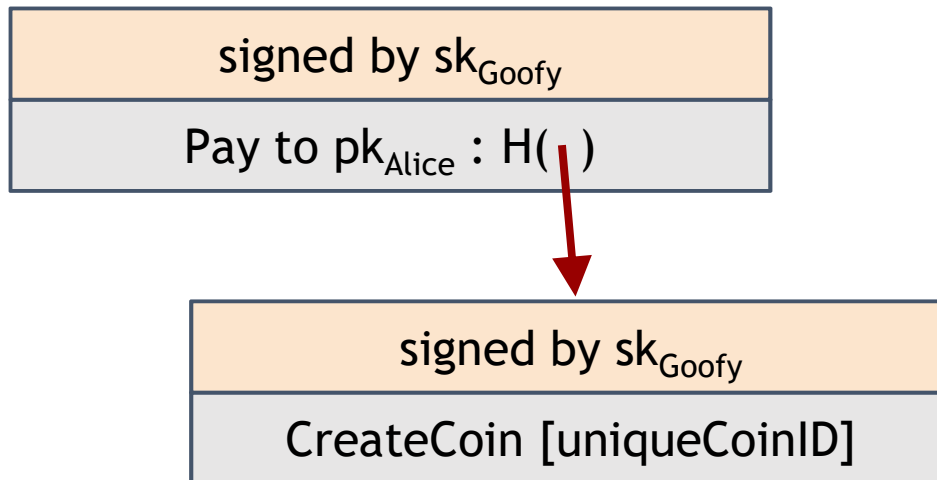
CreateCoin [uniqueCoinID]

New coins belong to me.



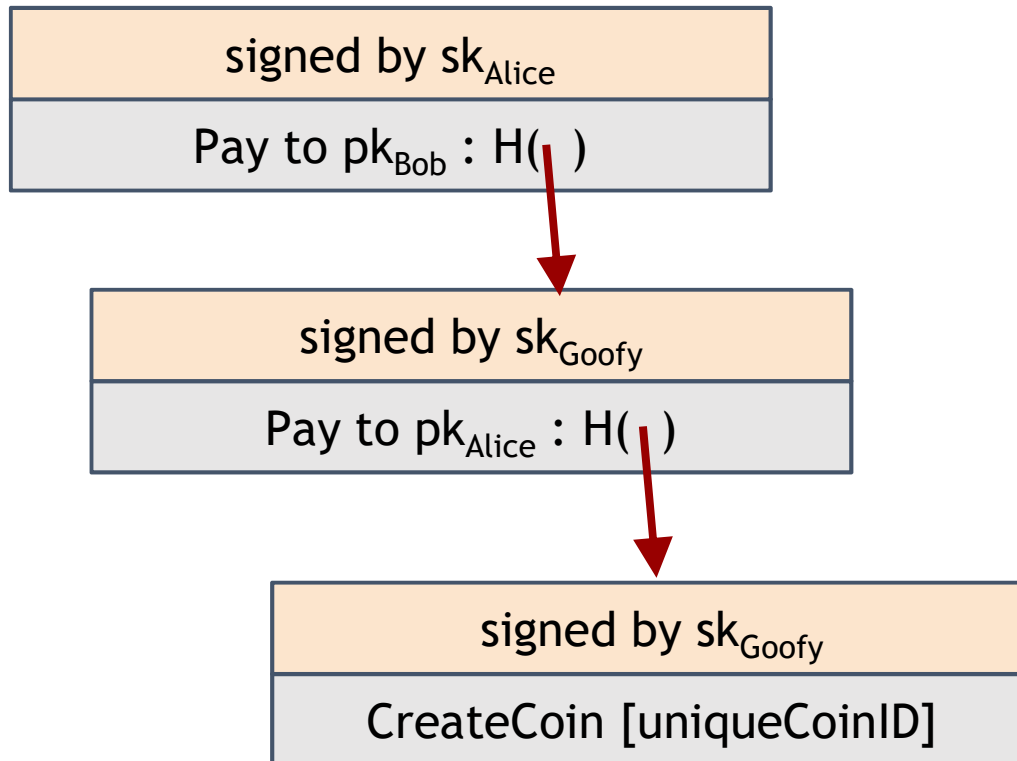
GoofyCoin – Spending Coins

A coin's owner can spend it.



GoofyCoin – Spending Coins

The recipient can pass on the coin again.



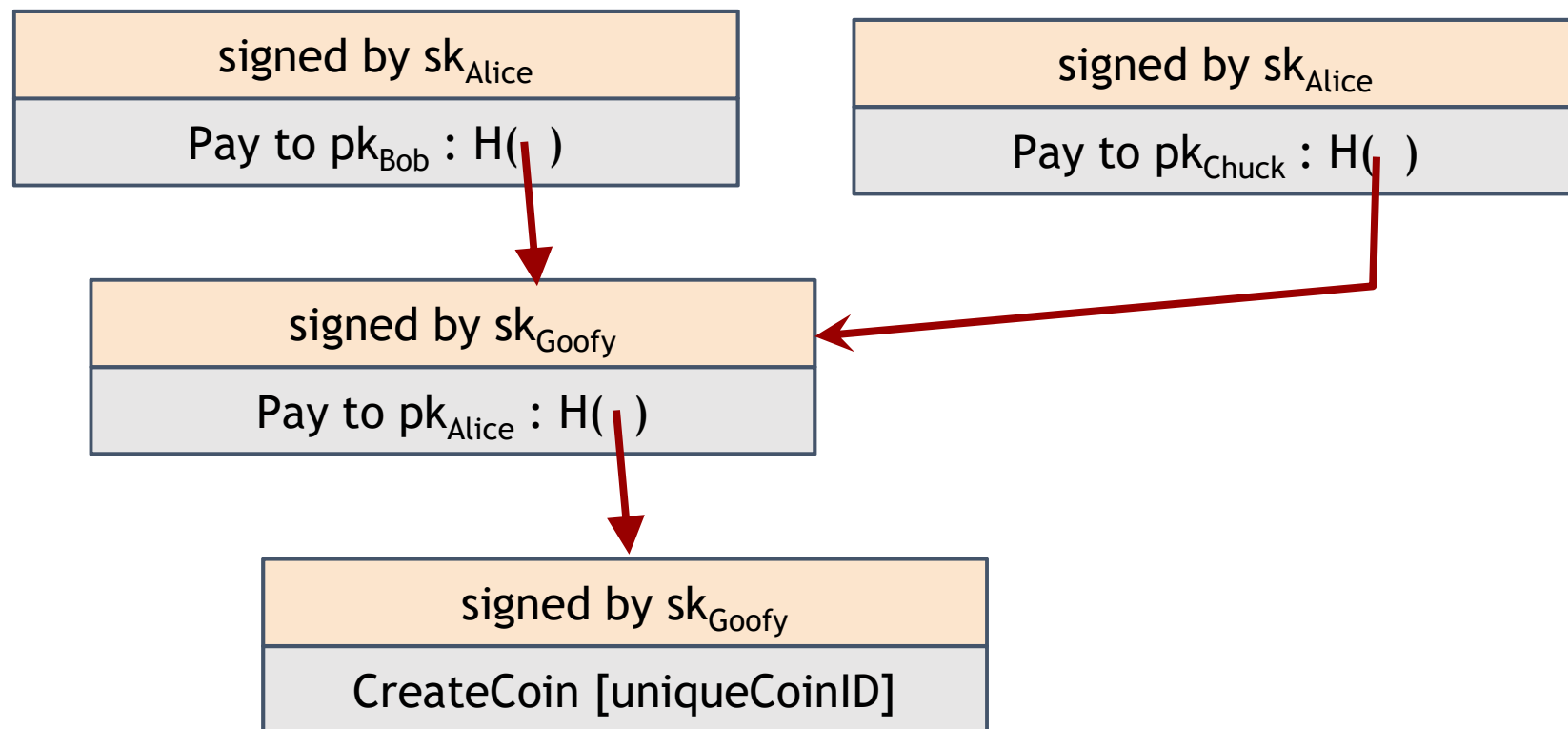
Bob owns it now.



To summarize, the rules of Goofycoin:

- Goofy can create new coins by simply signing a statement that he's making a new coin with a unique coin ID.
- Whoever owns a coin can pass it on to someone else by signing a statement that says, "Pass on this coin to X" (where X is specified as a public key).
- Anyone **can verify the validity of a coin by following the chain of hash pointers back to its creation by Goofy**, verifying all signatures along the way.

GoofyCoin – Double Spending Attack



GoofyCoin – Double Spending Attack

Double-spending attack

The main design challenge in digital currency

Goofycoin - Summary

- Goofycoin does not solve the double-spending attack, and therefore it's not secure.
- Goofycoin is simple, and its mechanism for transferring coins is actually **similar to that of Bitcoin**, but **because it is insecure**, it is inadequate as a cryptocurrency.



ScroogeCoin

ScroogeCoin

- To solve the double-spending problem, we'll design another cryptocurrency, called ScroogeCoin.
- ScroogeCoin is built off of GoofyCoin, but it's a bit more complicated in terms of data structures.
- The first key idea is that a **designated entity called Scrooge publishes an append-only ledger** containing the **history of all transactions**.
- The **append-only property** ensures that any **data written to this ledger** will **remain forever in the ledger**.

ScroogeCoin (2)

- If the ledger is truly append only, **we can use it to defend against double spending by requiring all transactions to be written in the ledger before they are accepted.**
- That way, it will be **publicly documented if coins were previously sent to a different owner.**
- **To implement this append-only functionality, Scrooge can build a block chain, which he will digitally sign.**

ScroogeCoin (3)

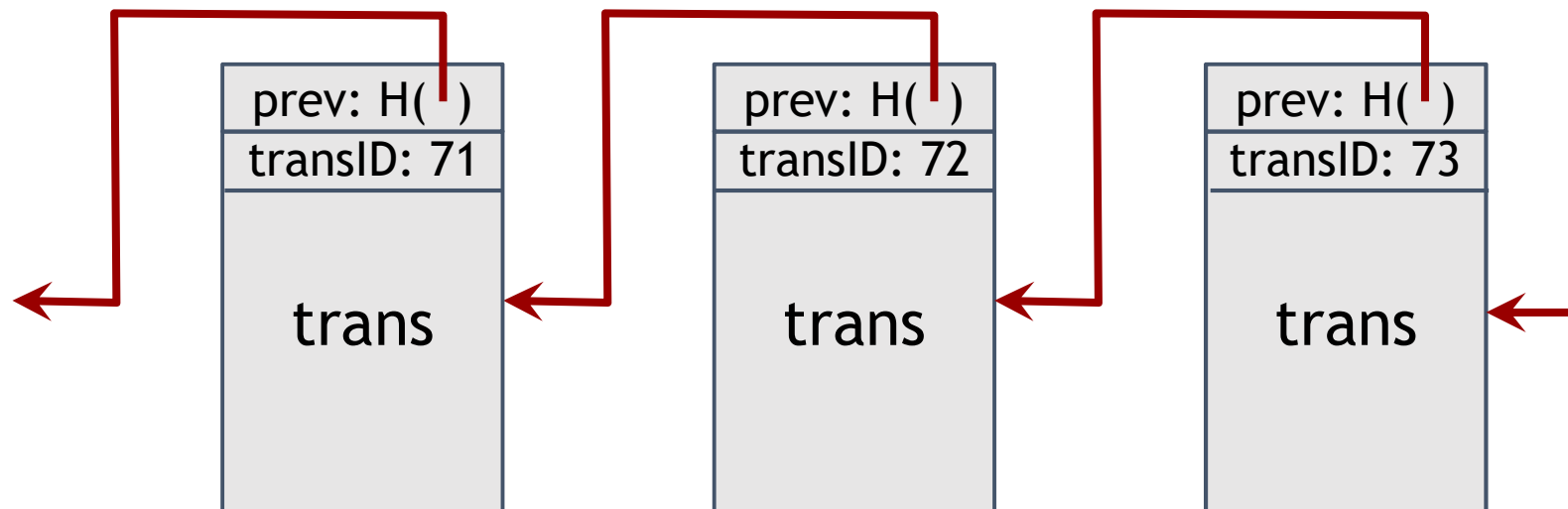
- It consists of a series of data blocks, each with one transaction in it (**in practice, as an optimization, we'd really put multiple transactions in the same block, as Bitcoin does.**)
- **Each block has the ID of a transaction, the transaction's contents, and a hash pointer to the previous block.**
- Scrooge **digitally signs the final hash pointer, which binds all the data in this entire structure, and he publishes the signature along with the block chain.**

ScroogeCoin

Scrooge publishes a history of all transactions
(a block chain, signed by Scrooge)



$H()$



optimization: put multiple transactions in the same block

ScroogeCoin (4)

- In Scroogecoin, **a transaction only counts if it is in the block chain signed by Scrooge.**
- Anybody can **verify that a transaction was endorsed by Scrooge by checking Scrooge's signature on the block** that records the transaction.
- Scrooge makes sure that he **doesn't endorse a transaction that attempts to double spend an already spent coin.**

ScroogeCoin (5)

- Why do we need a block chain with hash pointers in addition to having Scrooge sign each block? This ensures the append-only property.
- If Scrooge tries to add or remove a transaction, or to change an existing transaction, it will affect all following blocks because of the hash pointers.
- As long as someone is monitoring the latest hash pointer published by Scrooge, the change will be obvious and easy to catch.

ScroogeCoin (6)

- In a system where Scrooge signed blocks individually, you'd have to keep track of every single signature Scrooge ever issued.
- A **block chain makes it easy for any two individuals to verify** that they **have observed the same history of transactions signed by Scrooge**.
- In ScroogeCoin, there are two kinds of transactions.
- The **first kind is CreateCoins**, which is just like the operation Goofy could do in Goofycoin to make a new coin.
- With ScroogeCoin, we'll extend the semantics a bit to **allow multiple coins to be created in one transaction**.
- By definition, a **CreateCoins transaction is always valid if it is signed by Scrooge**.

ScroogeCoin

CreateCoins transaction creates new coins

transID: 73 type:CreateCoins		
coins created		
<i>num</i>	<i>value</i>	<i>recipient</i>
0	3.2	0x...
1	1.4	0x...
2	7.1	0x...

← coinID 73(0)

← coinID 73(1)

← coinID 73(2)

Valid, because I said so.



A CoinID is a combination of a transaction ID and the coin's serial number in that transaction.

ScroogeCoin (7)

- The second kind of transaction is PayCoins. It consumes some coins (i.e., destroys them) and creates new coins of the same total value.
- The new coins might belong to different people (public keys).
- This transaction has to be signed by everyone who's paying in a coin.
- So if you're **the owner of one of the coins that's going to be consumed in this transaction, then you need to digitally sign the transaction to say that you're OK with spending this coin.**

ScroogeCoin (8)

- The rules of ScroogeCoin say that the **PayCoins transaction is valid if it satisfies four conditions:**
 - The consumed coins are valid, that is, they were created in previous transactions.
 - The consumed coins have not already been consumed in some previous transaction. That is, this is not a double-spend transaction.
 - The total value of the coins that come out of this transaction is equal to the total value of the coins that went in. That is, only Scrooge can create new value.
 - The transaction is validly signed by the owners of all coins consumed in the transaction.

ScroogeCoin

transID: 73 type:PayCoins		
consumed coinIDs: 68(1), 42(0), 72(3)		
coins created		
<i>num</i>	<i>value</i>	<i>recipient</i>
0	3.2	0x...
1	1.4	0x...
2	7.1	0x...
signatures		

PayCoins transaction consumes
(and destroys) some coins,
and creates new coins of the same
total value

Valid if:

- consumed coins valid,
- not already consumed,
- total value out = total value in, and
- signed by owners of all consumed coins

ScroogeCoin

Immutable coins

- Coins can't be transferred, subdivided, or combined.
- How to overcome this problem?
 - You can get the effect of dividing coins by using transactions to subdivide: create new transaction to consume your coin and pay out two new coins to yourself

ScroogeCoin

Advantages?

- Prevents double spending
- Makes sure all transactions are valid, i.e., each coin is consumed only once

Problems?

- Scrooge has too much influence
- He cannot spend other peoples coins (can't create fake transactions) – why?
- However, he can stop endorsing other peoples transaction and deny them service
 - If he is greedy, he may also ask for a service/transaction fee for endorsing every transaction
- Also, as Scrooge is in charge of creating new coins, he can create as many for himself as he wants
- Last, he can get “bored” of the system and just stop updating the block chain altogether!

ScroogeCoin

Don't worry, I'm honest.



Crucial question:

Can we descroogify the currency, and operate without any central, trusted party?